杂项

1. 常用模板 1
1.1. 宏调试 1
2. 浮点数 2
3. 时间复杂度分析 2
4. 整数除法 2
5. 随机 3
6. 龟速乘 3
7. 常数优化 4
7.1. 快速取模 4
7.2. 快读 5
8. 离线算法 5
8.1. 莫队 5
8.1.1. 适用范围 5
8.1.2. 时间复杂度 6
8.1.3. 奇偶排序优化 6
8.1.4. 回滚莫队 8
8.1.4.1. 不删除莫队 8
8.1.4.2. 不添加莫队 12
8.1.5. 带修莫队 15
8.2. 整体二分 15
8.2.1. 离线区间询问 18
9. 置换环 20
10. 开根号 20
11. 矩阵旋转 20
12. etc 21
12.1. 环境配置 21

1. 常用模板

1.1. 宏调试

检测代码运行速度

```
int main() {
#define DEBUG
#ifdef DEBUG
    double begin_time = clock();
#endif
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
```

```
    int _ = 1;
    cin >> _;
    while (_--)
        solve();

#ifdef DEBUG
    double end_time = clock();
    cout << "\n----------\n";
    cout << fixed << setprecision(3);
    cout << (end_time - begin_time) / CLOCKS_PER_SEC << " s";
#endif
    return 0;
}
```

防止提交时忘记删除 freopen

---

2. 浮点数

输出x位小数

```
cout << setprecision(10) << endl;
```

不要直接用 cin 读入,cin 读入的效率很慢,可以用 string 读,然后 stod 转换

```
string s;
cin >> s;
db x = stod(s);
```

stod 如果是非法输入:

1.会自动截取最前面的浮点数,直到不满足浮点数为止

2.如果最前面不是小数点或数字,会 re

3.如果最前面是小数点,会自动转化在前面补零

stold:string to long double

3. 时间复杂度分析

- 势能分析法

4. 整数除法

上取整

```
i64 ceilDiv(i64 n, i64 m) {
    if (n >= 0) {
        return (n + m - 1) / m;
    } else {
        return n / m;
    }
}

i64 floorDiv(i64 n, i64 m) {
    if (n >= 0) {
        return n / m;
    } else {
        return (n - m + 1) / m;
    }
}
```

5. 随机

```
mt19937_64 rnd(chrono::duration_cast<chrono::nanoseconds>
(chrono::system_clock::now().time_since_epoch()).count());
void solve()
{
    ll l, r;
    cin >> l >> r;
    cout << rnd() << endl;
    cout << rnd() % (r - l + 1) + l << endl;
}
vector<int> a;
shuffle(a.begin(), a.end(), rnd);
```

6. 龟速乘

适用于两个 ll 级别的数乘法取模

```
ll qmul(ll a, ll b)
{
    ll ans = 0;
    while (b)
    {
        if (b & 1)
```

```
            ans = (ans + a) % mod;
        a = a * 2 % mod;
        b >>= 1;
    }
    return ans;
}
```

7. 常数优化

7.1. 快速取模

当模数是从输入中给定的

```
struct fastmod
{
    using u64 = uint64_t;
    using u128 = __uint128_t;
    int f, l;
    u64 m, d;
    fastmod(u64 d) : d(d)
    {
        l = 64 - __builtin_clzll(d - 1);
        const u128 one = 1;
        u128 M = ((one << (64 + l)) + (one << l)) / d;
        if (M < (one << 64))
            f = 1, m = M;
        else
            f = 0, m = M - (one << 64);
    }
    friend u64 operator/(u64 n, const fastmod &m)
    { // get n / d
        if (m.f)
            return u128(n) * m.m >> 64 >> m.l;
        else
        {
            u64 t = u128(n) * m.m >> 64;
            return (((n - t) >> 1) + t) >> (m.l - 1);
        }
    }
    friend u64 operator%(u64 n, const fastmod &m)
```

```
    { // get n % d
        return n - n / m * m.d;
    }
};
ull p;
cin >> p;
fastmod a(p);
ull n = 1e9 + 7;
cout << n % a;
```

7.2. 快读

```
int read()
{
    int x = 0, f = 1;
    char ch = getchar();
    while (ch < '0' || ch > '9')
    {
        if (ch == '-')
            f = -1;
        ch = getchar();
    }
    while (ch >= '0' && ch <= '9')
    {
        x = x * 10 + ch - 48;
        ch = getchar();
    }
    return x * f;
}
```

8. 离线算法

8.1. 莫队

8.1.1. 适用范围

- 询问区间信息
- 可以离线算法
- 难以高速合并区间
- 可以高速拓展到 l/r +−1

8.1.2. 时间复杂度

把整个区间分成$\frac{n}{B}$块,q 为询问数,n 为区间长度

将所有询问区间离线,排序(第一关键字,l/B,第一关键字,r)

左端点: B*q(每个块的左端点一次最多移动块长 B)

右端点:$\frac{n}{B} * n$(每个块的右端点加起来移动 n)

总时间复杂度:$O\left(B * q + \frac{n}{B} * n\right)$

令 B=$\frac{n}{\sqrt{q}}$,变成$O(n\sqrt{q})$

8.1.3. 奇偶排序优化

使右端点在不同块区间呈峰形移动

```
[&](array<ll, 3> a, array<ll, 3> b)
{
    int c=a[0]/B;
    if(c!=b[0]/B)
        return c<b[0]/B;
    return c%2?a[1]>b[1]:a[1]<b[1];
};
```

P1494 [国家集训队] 小 Z 的袜子

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define stop cout << "?" << '\n'
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<long long, long long> pll;
const int N = 5e4 + 10, M = 1e4 + 10;
const ll mod = 1e9 + 7;
const double pi = acos(-1);
int n, q;
ll c[N], cnt[N], ans[N], ans2[N];
array<ll, 3> que[N];
void solve()
{
```

```
    cin >> n >> q;
    for (int i = 1; i <= n; i++)
        cin >> c[i];
    for (int i = 1; i <= q; i++)
    {
        ll l, r;
        cin >> l >> r;
        que[i] = {l, r, i};
        ans2[i] = (r - l + 1) * (r - l) / 2;
    }
    int B = 500;
    sort(que + 1, que + q + 1, [&](array<ll, 3> a, array<ll, 3> b)
         {
        int c=a[0]/B;
        if(c!=b[0]/B)
        return c<b[0]/B;
        return c%2?a[1]>b[1]:a[1]<b[1]; });
    ll l = 1, r = 0, tmp = 0;
    auto add = [&](int x)
    {
        tmp += cnt[c[x]];
        cnt[c[x]]++;
    };
    auto del = [&](int x)
    {
        cnt[c[x]]--;
        tmp -= cnt[c[x]];
    };
    for (int i = 1; i <= q; i++)
    {
        while (r < que[i][1])
            r++, add(r);
        while (l > que[i][0])
            l--, add(l);
        while (r > que[i][1])
            del(r), r--;
        while (l < que[i][0])
            del(l), l++;
        ans[que[i][2]] = tmp;
```

```
    }
    for (int i = 1; i <= q; i++)
    {
        ll d = gcd(ans[i], ans2[i]);
        if (ans[i])
            cout << ans[i] / d << "/" << ans2[i] / d << endl;
        else
            cout << "0/1" << endl;
    }
}
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

8.1.4. 回滚莫队

对于普通莫队,如果添加/删除中的一个时间复杂度过高,导致普通莫队超时,就可以考虑回滚莫队

8.1.4.1. 不删除莫队

把询问按照左端点所在块为第一关键字,右端点为第二关键字

1.如果左右端点在一个块内,暴力求解

2.对于左端点在一个块的询问,右端点是递增的

设当前块为 T,当前询问的左端点为 qry[0],右端点为 qry[1]

(1)左指针 L 移动到 l[T+1],右指针 R 移动到 r[T]

(2)$R \to$qry[1]

设$l_1 = L, l_1 \to$qry[0]

(3)得到这次询问的答案

(4)回滚,$l_1 \to L$

https://www.luogu.com.cn/problem/P5906

区间中相同的数的最远间隔距离

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define sum(x) (a[x].sum)
#define tag(x) (a[x].t)
#define ls(x) (a[x].l)
#define rs(x) (a[x].r)
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef double db;
const ll mod = 998244353;
const int N = 2e5 + 100, M = 25;
void solve() {
    int n, m, res = 0;
    cin >> n;
    vector<int> a(n + 1);
    vector<int> b, bel(n + 1, 0), r(n + 1), l(n + 1);
    const int B = sqrt(n);
    int tot = n / B + 1;
    for (int i = 1; i <= tot; i++) {
        if (i * B > n)
            break;
        l[i] = (i - 1) * B + 1;
        r[i] = i * B;
    }
    if (r[tot] < n)
        tot++, l[tot] = r[tot - 1] + 1, r[tot] = n;

    for (int i = 1; i <= n; i++) {
        bel[i] = (i - 1) / B + 1;
        cin >> a[i];
        b.push_back(a[i]);
    }
```

```
    sort(b.begin(), b.end());
    b.erase(unique(b.begin(), b.end()), b.end());
    for (int i = 1; i <= n; i++)
        a[i] = lower_bound(b.begin(), b.end(), a[i]) - b.begin() + 1;
    cin >> m;

    vector<pii> cnt(n + 1), pre(n + 1); // cnt.x 左边 cnt.y 右边
    vector<array<int, 3>> qry(m);
    vector<int> ans(m);
    for (int i = 0; i < m; i++) {
        int l, r;
        cin >> l >> r;
        qry[i] = {l, r, i};
    }

    sort(qry.begin(), qry.end(), [&](array<int, 3> &a, array<int, 3> &b) {
        if (bel[a[0]] != bel[b[0]])
            return bel[a[0]] < bel[b[0]];
        return a[1] < b[1];
    });

    auto addr = [&](int x) -> void {
        cnt[a[x]].x ? cnt[a[x]].y = x : cnt[a[x]].x = cnt[a[x]].y = x;
        res = max(res, abs(cnt[a[x]].x - cnt[a[x]].y));
    };
    auto addl = [&](int x) -> void {
        pre[a[x]].y ? pre[a[x]].x = x : pre[a[x]].x = pre[a[x]].y = x;
        res = max(res, cnt[a[x]].y ? abs(pre[a[x]].x - cnt[a[x]].y) : abs(pre[a[x]].x - pre[a[x]].y));
    };
    auto del = [&](int x) -> void {
        cnt[a[x]] = {0, 0};
    };
    auto rb = [&](int x) -> void {
        pre[a[x]] = {0, 0};
```

```
    };
    int L = r[bel[qry[0][0]]] + 1;
    int R = r[bel[qry[0][0]]];
    int now = r[bel[qry[0][0]]];
    for (int i = 0; i < m; i++) {

        if (bel[qry[i][0]] == bel[qry[i][1]]) // 询问在同一块内,暴力求解
        {
            for (int j = qry[i][0]; j <= qry[i][1]; j++)
                pre[a[j]].x ? pre[a[j]].y = j : pre[a[j]].x = pre[a[j]].y = j;
            int tmp = 0;
            for (int j = qry[i][0]; j <= qry[i][1]; j++)
                tmp = max(tmp, abs(pre[a[j]].x - pre[a[j]].y));
            for (int j = qry[i][0]; j <= qry[i][1]; j++) // 还原
                pre[a[j]].x = pre[a[j]].y = 0;
            ans[qry[i][2]] = tmp;
            continue;
        }
        if (now ^ bel[qry[i][0]]) // 需要更新,新的块
        {
            // L=r[bel[qry[i][0]]] + 1  R=r[bel[qry[i][0]]]
            while (R > r[bel[qry[i][0]]])
                del(R), R--;
            while (L < r[bel[qry[i][0]]] + 1)
                del(L), L++;
            res = 0, now = bel[qry[i][0]];
        }
        while (R < qry[i][1])
            addr(++R);
        int tmp = res, l1 = L;
        while (l1 > qry[i][0])
            addl(--l1);
        ans[qry[i][2]] = res;
        while (l1 < L)
            rb(l1), l1++;
        res = tmp;
    }
```

```
    for (auto i : ans)
        cout << i << endl;
}
int main() {
    cout << setiosflags(ios::fixed) << setprecision(10);
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

8.1.4.2. 不添加莫队

把询问按照左端点所在块为第一关键字,右端点为第二关键字

1.如果左右端点在一个块内,暴力求解

2.对于左端点在一个块的询问,右端点是递增的

设当前块为 T,当前询问的左端点为 qry[0],右端点为 qry[1]

(1)左指针 L 移动到 l[T],右指针 R 移动到 n

(2)$R \to$qry[1]

设$l_1 = L, l_1 \to$qry[0]

(3)得到这次询问的答案

(4)回滚,$l_1 \to L$

https://www.luogu.com.cn/problem/P4137

区间 mex

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define sum(x) (a[x].sum)
#define tag(x) (a[x].t)
#define ls(x) (a[x].l)
#define rs(x) (a[x].r)
```

```
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef double db;
const ll mod = 998244353;
const int N = 2e5 + 100, M = 25;
void solve() {
    int n, m, res = 0;
    cin >> n >> m;
    const int B = sqrt(m) + 1;
    vector<int> a(n + 1), bel(n + 1), r(n + 1), l(n + 1);
    vector<int> cnt(N, 0), cnt1(N, 0);
    vector<array<int, 3>> qry(m);
    vector<int> ans(m);

    int tot = n / B;
    for (int i = 1; i <= tot; i++) {
        if (i * B > n)
            break;
        l[i] = (i - 1) * B + 1;
        r[i] = i * B;
    }
    if (r[tot] < n)
        tot++, l[tot] = r[tot - 1] + 1, r[tot] = n;
    for (int i = 1; i <= n; i++) {
        bel[i] = (i - 1) / B + 1;
        cin >> a[i];
    }

    for (int i = 1; i <= n; i++)
        cnt[a[i]]++;
    while (cnt[res])
        res++;

    for (int i = 0; i < m; i++) {
        int l, r;
        cin >> l >> r;
        qry[i] = {l, r, i};
```

```
    }
    sort(qry.begin(), qry.end(), [&](array<int, 3> &a, array<int, 3> &b) {
        if (bel[a[0]] != bel[b[0]])
            return bel[a[0]] < bel[b[0]];
        return a[1] > b[1];
    });

    auto add = [&](int x) -> void {
        cnt[a[x]]++;
    };
    auto del = [&](int x) -> void {
        cnt[a[x]]--;
        if (!cnt[a[x]])
            res = min(res, a[x]);
    };
    int L = 1, R = n, now = 0;
    for (int i = 0; i < m; i++) {
        if (bel[qry[i][0]] == bel[qry[i][1]]) {
            for (int j = qry[i][0]; j <= qry[i][1]; j++)
                cnt1[a[j]]++;
            int tmp = 0;
            while (cnt1[tmp])
                tmp++;
            for (int j = qry[i][0]; j <= qry[i][1]; j++)
                cnt1[a[j]]--;
            ans[qry[i][2]] = tmp;
            continue;
        }
        if (now != bel[qry[i][0]]) {
            while (R < n)
                add(++R);
            while (L < l[bel[qry[i][0]]])
                del(L++);
            int tmp = 0;
            while (cnt[tmp])
                tmp++;
            res = tmp;
            now = bel[qry[i][0]];
```

```
        }
        while (R > qry[i][1])
            del(R--);
        int tmp = res, l1 = L;
        while (l1 < qry[i][0])
            del(l1++);
        ans[qry[i][2]] = res;
        while (l1 > L)
            add(--l1);
        res = tmp;
    }
    for (auto i : ans)
        cout << i << endl;
}
int main() {
    cout << setiosflags(ios::fixed) << setprecision(10);
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

8.1.5. 带修莫队

P2709 小 B 的询问

8.2. 整体二分

[模板]可持久化线段树 2

给定一个数组 a,有 q 个询问，每次询问区间[l,r]内的第 k 小

先考虑只有一个询问,二分答案,check 是否有 k-1 个小于等于的数

如果有多个询问,则需要引入整体二分

```
struct BIT
{
    int n;
    vector<ll> a;
```

```
    BIT(int _n) : n(_n + 3), a(n + 1) {}
    int lb(int x) { return x & -x; }
    void add(int x, int y)
    {
        for (; x < n; x += lb(x))
            a[x] += y;
    }
    ll query(int x)
    {
        ll res = 0;
        for (; x; x ^= lb(x))
            res += a[x];
        return res;
    }
};
struct node
{
    ll x, y, k, id;
    bool typ;
};
void solve()
{
    int n, m, tp = 0;
    cin >> n >> m;
    BIT bit(n);
    vector<node> q(n + m + 1), q1(n + m + 1), q2(n + m + 1);
    vector<ll> a(n + 1), ans(m + 1);
    for (int i = 1; i <= n; i++)
    {
        cin >> a[i];
        q[++tp] = {a[i], 1, inf, i, 0};
    }
    for (int i = 1; i <= m; i++)
    {
        ll l, r, k;
        cin >> l >> r >> k;
        q[++tp] = {l, r, k, i, 1};
    }
    auto dfs = [&](const auto &dfs, ll l, ll r, ll L, ll R) -> void
```

```
    {
        if (l > r)
            return;
        if (L == R)
        {
            for (ll i = l; i <= r; i++)
                if (q[i].typ)
                    ans[q[i].id] = L;
            return;
        }
        ll mid = (L + R) >> 1, cnt1 = 0, cnt2 = 0;
        for (ll i = l; i <= r; i++)
        {
            if (!q[i].typ)
            {
                if (q[i].x <= mid)
                {
                    bit.add(q[i].id, 1);
                    q1[++cnt1] = q[i];
                }
                else
                    q2[++cnt2] = q[i];
            }
            else
            {
                ll res = bit.query(q[i].y) - bit.query(q[i].x - 1);
                if (res >= q[i].k)
                    q1[++cnt1] = q[i];
                else
                {
                    q[i].k -= res;
                    q2[++cnt2] = q[i];
                }
            }
        }
        for (ll i = 1; i <= cnt1; i++)
            if (!q1[i].typ)
                bit.add(q1[i].id, -1);
        for (ll i = 1; i <= cnt1; i++)
```

```
            q[i + l - 1] = q1[i];
        for (ll i = 1; i <= cnt2; i++)
            q[i + l + cnt1 - 1] = q2[i];

        dfs(dfs, l, l + cnt1 - 1, L, mid);
        dfs(dfs, l + cnt1, r, mid + 1, R);
    };
    dfs(dfs, 1, tp, -inf, inf);
    for (int i = 1; i <= m; i++)
        cout << ans[i] << endl;
}
```

8.2.1. 离线区间询问

- 区间颜色数

P1972 [SDOI2009] HH 的项链

```
struct BIT
{
    int n;
    vector<ll> a;
    BIT(int _n) : n(_n + 3), a(n + 1) {}
    int lb(int x) { return x & -x; }
    void add(int x, int y)
    {
        for (; x < n; x += lb(x))
            a[x] += y;
    }
    ll query(int x)
    {
        ll res = 0;
        for (; x; x ^= lb(x))
            res += a[x];
        return res;
    }
};
struct query
{
    int l, r, id;
    bool operator<(const query &T) const { return r < T.r; };
```

```
};
void solve()
{
    int n, m;
    cin >> n;
    BIT bit(n);
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    cin >> m;
    vector<query> q(m);
    vector<int> ans(m), mp(N, 0);
    for (int i = 0; i < m; i++)
    {
        cin >> q[i].l >> q[i].r;
        q[i].id = i;
    }
    sort(q.begin(), q.end());
    int pos = 1;
    for (auto [l, r, id] : q)
    {
        for (int i = pos; i <= r; i++)
        {
            if (mp[a[i]])
                bit.add(mp[a[i]], -1);
            bit.add(i, 1);
            mp[a[i]] = i;
        }
        pos = r + 1;
        ans[id] = bit.query(r) - bit.query(l - 1);
    }
    for (auto i : ans)
        cout << i << endl;
}
```

- 区间 mex

P4137 Rmq Problem / mex

```
1
```

9. 置换环

应用:

求将一个排列变有序需要的最少操作次数

对每个节点，将其指向其排序后应该放到的位置，这样首尾相接形成了多个环。

答案为$\sum_{i=1}^{k} sz[i]-1$,即所有环的长度-1 求和

```
for (int i = 1; i <= n; i++)
{
    if(st[i])
        continue;
    int res = 0;//置换环的大小
    int x = i;
    while (!st[x])
    {
        st[x] = true;
        x = to[x];
        res++;
    }
    ans += res - 1;
}
```

10. 开根号

```
ll safe_sqrt(long long a)
{
    ll ret = sqrt(a);
    while ((ret + 1) * (ret + 1) <= a)
        ret++;
    while (ret * ret > a)
        ret--;
    return ret;
}
```

11. 矩阵旋转

顺时针旋转90°

```
void Rotate()
{
```

```
    for (int i = 1; i <= (n - 1) / 2 + 1; i++)
        for (int j = 1; j <= n / 2; j++)
        {
            int num = a[i][j];
            a[i][j] = a[n - j + 1][i];
            a[n - j + 1][i] = a[n - i + 1][n - j + 1];
            a[n - i + 1][n - j + 1] = a[j][n - i + 1];
            a[j][n - i + 1] = num;
        }
}
```

12. etc

1.深呼吸，不要紧张，慢慢读题，读明白题，题目往往比你想的简单。

2.暴力枚举:枚举什么，是否可以使用一些技巧加快枚举速度（预处理、前缀和、数据结构、数论分块）。

3.贪心:需要排序或使用数据结构（pq）吗，这么贪心一定最优吗。

4.二分：满足单调性吗，怎么二分，如何确定二分函数返回值是什么。

5.位运算：按位贪心，还是与位运算本身的性质有关。

6.数学题：和最大公因数、质因子、取模是否有关。

7.dp：怎么设计状态，状态转移方程是什么，初态是什么，使用循环还是记搜转移。

8.搜索：dfs 还是 bfs ，搜索的时候状态是什么，需要记忆化吗。

9.树上问题：是树形 dp、树上贪心、或者是在树上搜索。

10.图论：依靠什么样的关系建图，是求环统计结果还是最短路。

11.组合数学：有几种值，每种值如何被组成，容斥关系是什么。

12.交互题：log(n)次如何二分，2*n 次如何通过 n 次求出一些值，再根据剩余次数求答案。

13.如果以上几种都不是，多半是有一个 point 你没有注意到，记住正难则反。

12.1. 环境配置

vsc

1. format on save
2. run before save
3. Save before run

4. 编译命令

编译,运行,预编译命令

g++ A.cpp -o A -Ddx3ea ; .A

4.coderunner

拓展-> coderunner ->拓展设置 -> setting.json

```
"cpp": "cd $dir && g++ $fileName -o $fileNameWithoutExt -Ddx3ea && $dir$fileNameWithoutExt",
```

Clear Previous Output

Save File Before Run

```
#ifdef dx3ea
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
#endif
```